Open Source Agents on AWS

Strands Framework · Open Protocols · AgentCore Runtime

Ryan Cormack · AWS, Agentic Engineering

Who has deployed an AI agent in the last 6 months?

Keep your hand up if you felt confident about how it integrates with other systems...

Ryan Cormack

Building AI things on AWS

Building agents · Contributing to open source



The Problem

Models are commoditising. Swap Claude, Nova, Llama freely.

But the plumbing — how agents use tools, talk to each other, interact with users, get hosted — is a mess of proprietary SDKs.

Today: a stack built entirely on open standards.



Tools



Communication



Hosting

01

THE AGENT HARNESS

Strands Framework

What is an Agent?

At its core, an agent is just a loop:

1. Ask the model

2. Tool call?
   Execute it, feed result back

3. No tool call?
   We're done.

```
while (true) {
  const resp = model.invoke(messages, tools)

  if (resp.hasToolCalls) {
    const result = executeTool(resp.toolCall)
    messages.push(result)
  } else {
    return resp  // done
  }
}
```

The harness doesn't make decisions — it structures them.

Strands Framework

```
npm install @strands-agents/sdk
```

Open source · TypeScript & Python · Apache 2.0

Model-Agnostic

Bedrock, OpenAI, Anthropic, Google + more

Tool-First

Functions with types = tools

Minimal

No chains, no graphs, no DSL

Extensible

Protocols plug in as tools

Batteries Included: Strands Harness

The pre-assembled harness on top of the same SDK — one import, benchmarked defaults

```
import { createHarness } from '@strands-agents/harness'
const agent = await createHarness()
```

Tuned defaults

System prompt, tools, memory, sessions, context management — benchmarked, not guessed

28% lower cost

Same accuracy as Claude Code & Codex across six benchmarks

Batteries included

Shell, file (read/write/edit), and web tools out of the box

Smart context mgmt

Truncates bulky results, compacts at 85% context, recovers from overflow

Memory & sessions

Long-term memory across runs; resume by session ID

No lock-in

Every default overridable — you get back a standard Strands Agent, not a wrapper

This space moves fast. For many scenarios, batteries included beats hand-rolled.

The Simplest Strands Agent

Define a tool

tool() with name, description, inputSchema (Zod), callback

Create an agent

new Agent({ tools: [...] })

Invoke with

await agent.invoke()

```
import { Agent, tool } from '@strands-agents/sdk'
import { z } from 'zod'

const getWeather = tool({
  name: 'get_weather',
  description: 'Get weather for a city',
  inputSchema: z.object({
    city: z.string().describe('City name'),
  }),
  callback: ({ city }) =>
    `22° C, sunny in ${city}`,
})

const agent = new Agent({ tools: [getWeather] })
await agent.invoke('Weather in Manchester?')
```

02

AGENT TOOLS

Model Context Protocol (MCP)

Why MCP Matters

Every framework defines tools differently.

A tool built for one doesn't work in another.

Build Once	Teams build agnostic tools over MCP. No framework lock-in.
Expose Everywhere	One tool server exposed to every agent in your org.
Reach End Users	Same MCP tools power internal agents and customer-facing products.

MCP: What It Exposes



Tools

Callable functions
with schemas



Resources

Data the agent
can read



Prompts

Reusable prompt
templates

All exposed over JSON-RPC 2.0

MCP Request Flow



Discover

`tools/list`

Client asks for available tools

Receive

`→ definitions`

Server returns names, schemas, descriptions

Invoke

`tools/call`

Client calls a tool with arguments

Result

`→ content[]`

Server returns text, images, resources

```
// tools/list response
{ "tools": [{ "name": "get_weather",
    "inputSchema": { "properties":
      { "city": { "type": "string" } } } }] }
```

MCP Transports

stdio (local)

```
import { McpClient } from
  '@strands-agents/sdk/mcp'
import { StdioClientTransport }
  from '@modelcontextprotocol/sdk'

const mcp = new McpClient({
  transport: new StdioClientTransport({
    command: 'uvx',
    args: ['awslabs.aws-docs...'],
  }),
})
```

Spawns a child process. Fast.

Streamable HTTP (remote)

```
import { McpClient } from
  '@strands-agents/sdk/mcp'
import { StreamableHTTPClient
  Transport } from
  '@modelcontextprotocol/sdk'

const mcp = new McpClient({
  transport:
    new StreamableHTTPClientTransport(
      new URL('https://mcp.acme.co')
    ),
})
```

Standard HTTP. Load-balanceable.

MCP + Strands

Zero tool definitions

Multiple servers

Local + remote mixed

Runtime discovery

Composable

```
import { Agent } from '@strands-agents/sdk'
import { McpClient } from '@strands-agents/sdk/mcp'

const github = new McpClient({
  transport: new StdioClientTransport({ command: 'uvx',
    args: ['@mcp/server-github'] }),
})

const jira = new McpClient({
  transport: new StreamableHTTPClientTransport(
    new URL('https://mcp.internal/jira')
  ),
})

const agent = new Agent({
  tools: [github, jira],  // compose!
})
```

MCP Updates

2026-07-28 Specification

BEFORE

- initialize / initialized handshake
- Mcp-Session-Id header
- Sticky sessions required
- No standard caching

AFTER

- Self-describing requests (_meta)
- No session state in transport
- Round-robin load balancers work
- Cacheable results (ttlMs)
- Routable headers (Mcp-Method)
- OTel trace context in spec

Gateways route on Mcp-Method + Mcp-Name headers — no JSON body parsing needed

03

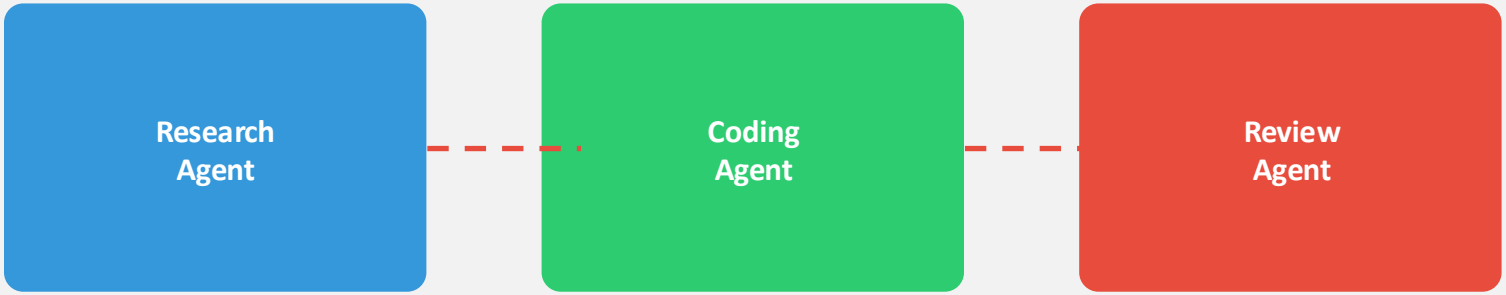
AGENTS WORKING WITH AGENTS

Agent2Agent (A2A)

The Multi-Agent Problem

Different teams, different frameworks.

Today, agents can't talk to each other.



A2A solves this — HTTP-based, framework-agnostic.

A2A Protocol

Agent Cards

JSON manifest at /.well-known/agent.json describing what an agent can do

Tasks

Units of work one agent sends to another with structured input

Messages & Artifacts

Structured communication and rich outputs (code, files, data)

Streaming

Server-Sent Events for real-time progress on long-running tasks

REST-native · Any language · Any framework

A2A Flow



Discover

`GET /.well-known/agent.json`

Fetch the Agent Card

Send Task

`POST /tasks`

Send work to agent

Stream

`SSE updates`

Progress streamed back

Result

`→ artifacts[]`

Structured outputs

All over HTTP. No shared memory, frameworks, or language.

Strands + A2A

A2AAgent connects to any remote A2A server

Wrap in tool() to expose as an orchestrator tool

The callback calls .invoke() on the remote agent

```
import { Agent, tool } from '@strands-agents/sdk'
import { A2AAgent } from '@strands-agents/sdk/a2a'
import { z } from 'zod'

const reviewAgent = new A2AAgent({
  url: 'http://review-service:9000',
})

const review = tool({
  name: 'code_review',
  description: 'Review code for issues',
  inputSchema: z.object({ code: z.string() }),
  callback: async ({ code }) => {
    const r = await reviewAgent.invoke(code)
    return String(r.lastMessage.content[0])
  },
})

const agent = new Agent({ tools: [review] })
```

04

CLIENTS FOR AGENTS

ACP

ACP — Agent Client Protocol

Local · Fast · Streaming

Discovery

Find available agents locally

Invocation

Send tasks with context

Streaming

Incremental results back

Lifecycle

Start, stop, manage agents

IDEs · CLIs · Desktop apps — write once, any ACP client can use it.

ACP v2: Going Remote

ACP v1 is stdio only. v2 adds HTTP + WebSocket for remote agents.

Streamable HTTP

POST for client → server messages
GET for long-lived SSE streams
Requires HTTP/2
POST returns 202 immediately;
responses come on the SSE stream

WebSocket

Full-duplex on same /acp endpoint
GET with Upgrade: websocket
Low-latency bidirectional
Same JSON-RPC messages
Same ACP lifecycle

Same protocol, local or remote. Agents built for IDEs can also run as cloud services.

Single /acp endpoint · Clients must support both profiles · Targeted for v1 as additive + v2 as baseline

04

CLIENTS FOR AGENTS

AG UI

AG UI: The Event Stream

16 typed events. Agent emits, any client consumes.

Lifecycle	`RUN_STARTED → STEP_* → RUN_FINISHED`
Text	`START → CONTENT (delta) → END`
Tool Calls	`START → ARGS → END`
State	`SNAPSHOT + DELTA (JSON Patch)`

```
// Every event inherits BaseEvent
interface BaseEvent {
  type: EventType  // 16 types
  timestamp?: number
  rawEvent?: any
  metadata?: Record<string, any>
}

// Core interface
type RunAgent =
  (input: RunAgentInput) =>
    Observable<BaseEvent>
```

Clients: CopilotKit · Kotlin · Java · Go · Rust · .NET · Dart · CLI · Slack/Teams

CopilotKit is one client. The protocol is open — any app that reads the event stream is a client.

AG UI + Strands (TypeScript)

```
import { Agent } from '@strands-agents/sdk'
import { StrandsAgentAdapter,
  createStrandsApp }
  from 'ag-ui-strands-adapter'

const agent = new Agent({
  model, tools, systemPrompt
})

const adapter = new StrandsAgentAdapter({
  agent,
  name: 'my_agent',
  description: 'Does things',
})

const app = createStrandsApp(adapter)
app.listen(8001)
```

```
// Any AG UI client — no CopilotKit needed
import { HttpAgent } from '@ag-ui/client'

const agent = new HttpAgent({
  url: 'http://localhost:8001',
})

agent.runAgent({ tools, context })
  .subscribe({
    next: (event) => {
      if (event.type ===
'TEXT_MESSAGE_CONTENT')
        process.stdout.write(event.delta)
    }
  })
```

Client: any AG UI consumer

What We've Covered

Strands	The open source agent harness
MCP	Open tool protocol — tools/list, tools/call
A2A	Agent-to-agent over HTTP
ACP	JSON-RPC protocol for IDE ↔ agent communication
AG UI	Standard event stream for frontends

Now: how do we run this in production?

05

HOSTING AGENTS

AgentCore Runtime & Gateway

The Operational Challenge

Your agent works on your laptop. Now what?

Scale

Bursty workloads

Sessions

State across requests

Observability

What did it do?

Security

IAM, secrets, isolation

Cost

Loops are expensive

→ AgentCore

Purpose-built runtime

AgentCore Runtime

Your agent is open source.

The runtime is managed.

Serverless

Scales to zero

Sessions

Persistent state

Auth

IAM integration

Guardrails

Token limits, safety

Bedrock

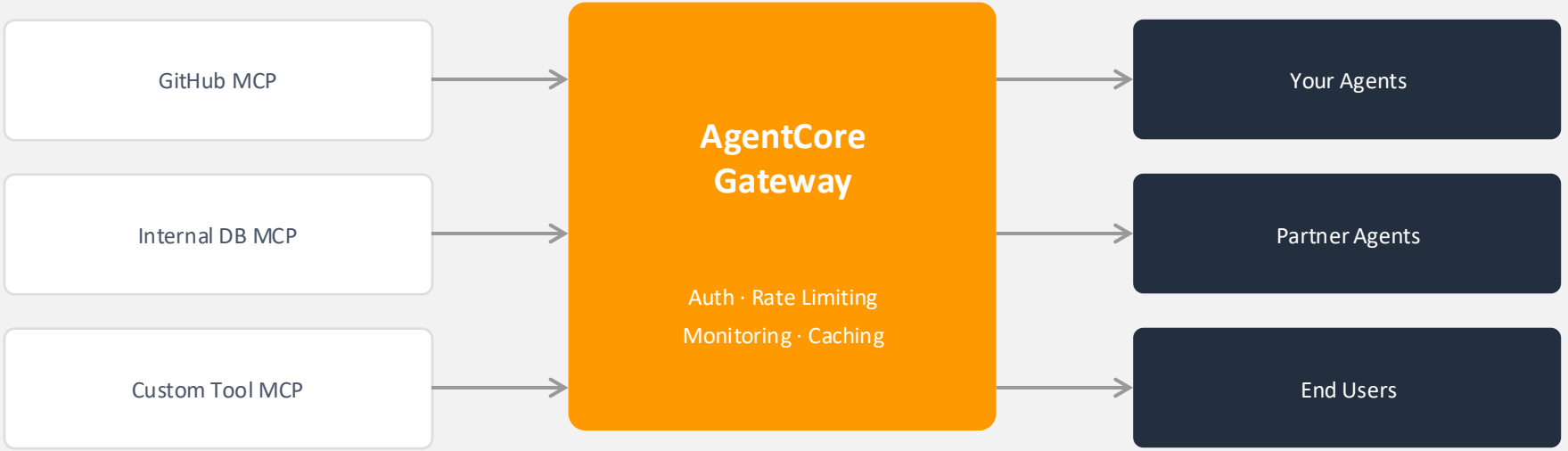
Native model access

Same Code

Laptop to production

AgentCore Gateway

Expose MCP servers as managed endpoints



Observability — OpenTelemetry

Built-in OTel instrumentation:

- Agent loop iterations
- Tool calls (latency, success/fail)
- Model invocations (tokens, cost)

Managed (AgentCore)

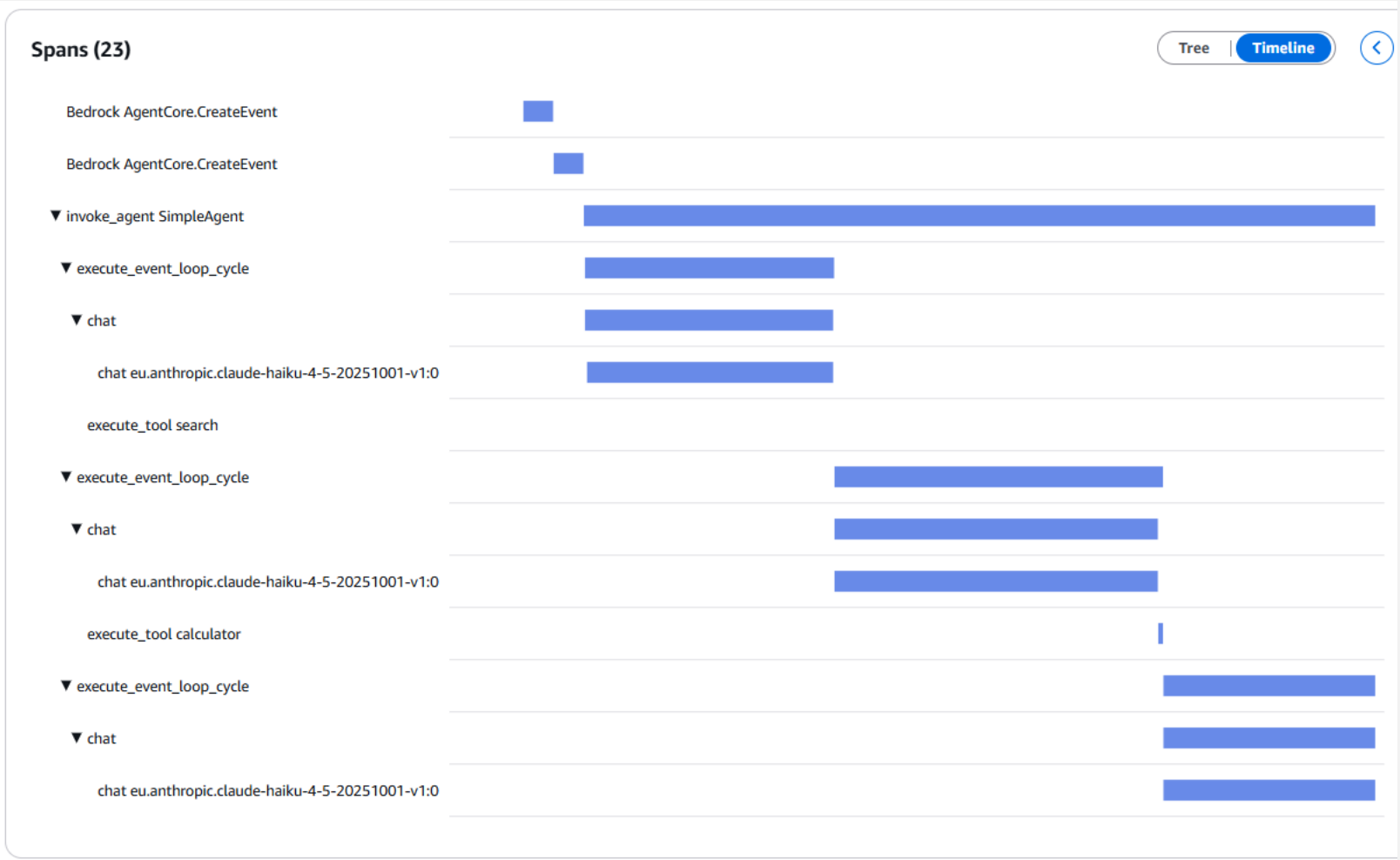
OTel → CloudWatch

Traces • Metrics • Logs

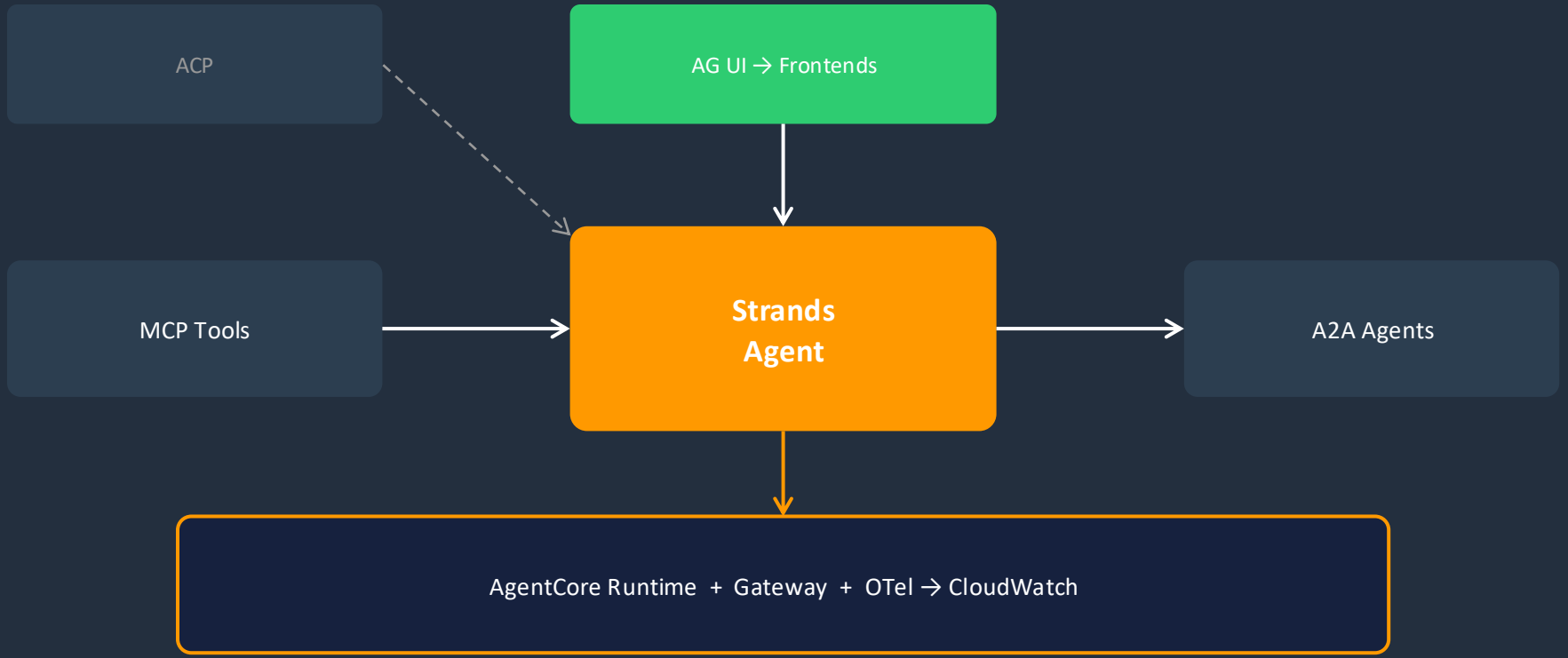
Self-hosted

OTel → Jaeger / Datadog / Honeycomb

Same instrumentation, your backend

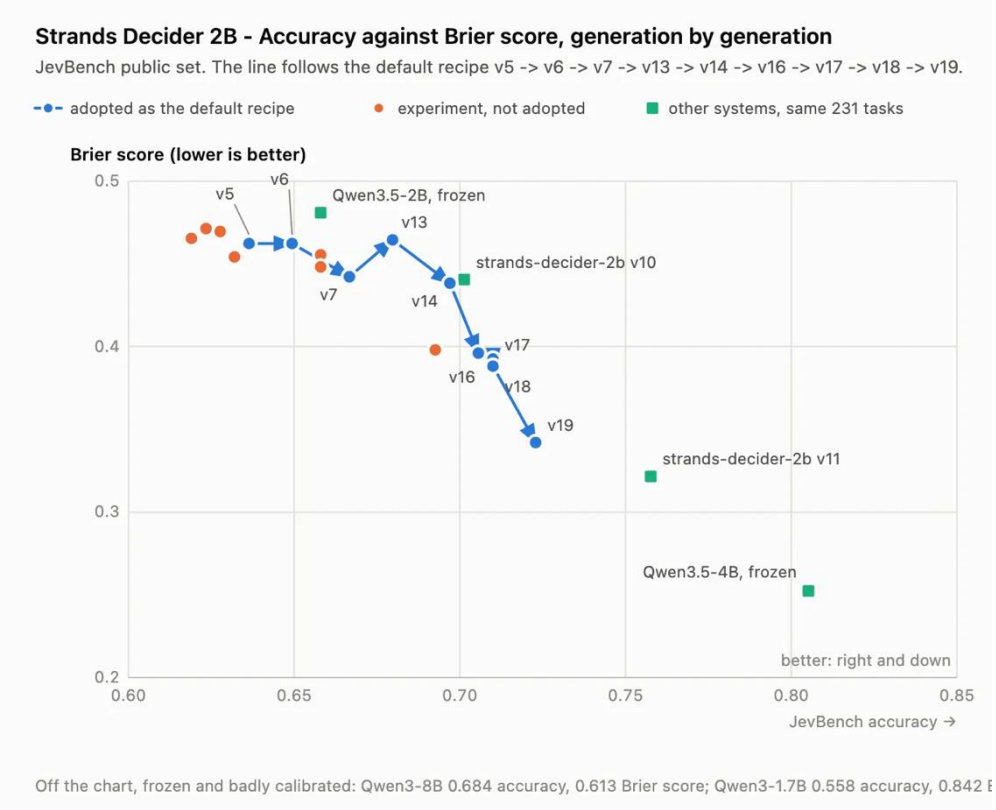

Spans (23)
Tree
Timeline
Bedrock AgentCore.CreateEvent
Bedrock AgentCore.CreateEvent
▼ invoke_agent SimpleAgent
▼ execute_event_loop_cycle
▼ chat
chat eu.anthropic.claude-haiku-4-5-20251001-v1:0
execute_tool search
▼ execute_event_loop_cycle
▼ chat
chat eu.anthropic.claude-haiku-4-5-20251001-v1:0
execute_tool calculator
▼ execute_event_loop_cycle
▼ chat
chat eu.anthropic.claude-haiku-4-5-20251001-v1:0

The Full Picture



Every piece: open protocol or open source. Every piece: swappable.

Decisions with Strands



Get Started Today

1. `npm install @strands-agents/sdk`
   Start building

2. `Connect an MCP server`
   10x more capable, zero tool code

3. `Deploy to AgentCore`
   Laptop to production in minutes

Thank you.



Strands
Agents

Ryan Cormack

Building agents · Contributing to open source

